MODUL 02 · BAB I · DASAR ALGORITMA

Algoritma Iteratif dan Rekursif: Studi Kasus Binary Search

Dua cara mengulang pekerjaan: dengan perulangan atau dengan fungsi yang memanggil dirinya sendiri.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Membedakan pendekatan iteratif dan rekursif beserta kelebihan dan kekurangannya.
- Mengidentifikasi kasus dasar (*base case*) dan kasus rekursif pada sebuah fungsi rekursif.
- Menjelaskan cara kerja Binary Search serta syarat yang harus dipenuhi masukannya.
- Menelusuri Binary Search versi iteratif maupun rekursif, termasuk tumpukan pemanggilan fungsi.
- Menjelaskan mengapa Binary Search membutuhkan paling banyak $\lfloor \lg n \rfloor + 1$ perbandingan.

PRASYARAT

- Modul 01: pseudocode, struktur percabangan dan perulangan, penelusuran algoritma.

BAB / MATERI ASAL

PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Algoritma Iteratif dan Rekursif)

POSISI DALAM URUTAN BELAJAR

Modul kedua. Konsep rekursi dipakai lagi pada Divide and Conquer (Modul 08), Recurrence (Modul 09), DFS (Modul 18), dan PRINT-LCS (Modul 12).

DAFTAR ISI

1. Dua Cara Mengulang Pekerjaan 1
2. Algoritma Iteratif dan Rekursif 2
3. Binary Search 3
4. Penelusuran Binary Search 5
5. Seberapa Cepat Binary Search? 7
6. Kesimpulan 8
- Kuis Pemahaman 10
- Kunci Jawaban dan Pembahasan 11
- Referensi 11

PENDAHULUAN

1. Dua Cara Mengulang Pekerjaan

Banyak masalah diselesaikan dengan mengulang pekerjaan yang serupa. Menjumlahkan seratus bilangan berarti mengulang operasi "tambahkan bilangan berikutnya" seratus kali. Mencari nama di buku telepon berarti berulang kali membuka halaman tengah lalu memilih separuh mana yang perlu dicari lagi.

Ada dua cara menuliskan pengulangan dalam algoritma. Cara pertama adalah **iteratif**: menggunakan struktur perulangan seperti `for` atau `while`. Cara kedua adalah **rekursif**: sebuah fungsi menyelesaikan masalah dengan memanggil dirinya sendiri untuk masalah yang lebih kecil. Keduanya sama kuatnya, artinya

setiap algoritma rekursif dapat ditulis ulang secara iteratif dan sebaliknya. Namun cara berpikir, kemudahan penulisan, dan pemakaian memorinya berbeda.

Modul ini membahas kedua pendekatan melalui satu studi kasus yang sangat penting: **Binary Search** (pencarian biner), yaitu algoritma untuk mencari sebuah bilangan di dalam array yang sudah terurut. Binary Search juga menjadi perkenalan pertama kita pada algoritma dengan waktu logaritmik, salah satu laju pertumbuhan paling efisien yang akan dibahas di Modul 05.

PEMBAHASAN INTI

2. Algoritma Iteratif dan Rekursif

2.1 Pendekatan iteratif

DEFINISI

Algoritma iteratif adalah algoritma yang menggunakan perulangan (iterasi) untuk mengulangi langkah-langkah penyelesaian masalah. Keadaan (*state*) perhitungan disimpan di dalam variabel yang nilainya diperbarui pada setiap putaran.

Algoritma `JUMLAH_n_BIL` di Modul 01 adalah contoh algoritma iteratif. Variabel `jumlah` menyimpan hasil sementara, dan perulangan `for` memperbaruinya satu per satu.

2.2 Pendekatan rekursif

DEFINISI

Algoritma rekursif adalah algoritma yang di dalamnya terdapat fungsi yang **memanggil dirinya sendiri** untuk menyelesaikan masalah yang lebih kecil dari masalah asalnya.

Agar rekursi tidak berjalan selamanya (ingat karakteristik *finiteness* di Modul 01), setiap fungsi rekursif yang benar harus memiliki tiga unsur berikut.

1. **Kasus dasar (*base case*)**: kondisi ketika masalah sudah cukup kecil sehingga jawabannya langsung diketahui tanpa memanggil diri sendiri.
2. **Kasus rekursif**: bagian yang memanggil fungsi yang sama untuk masalah yang lebih kecil.
3. **Kemajuan menuju kasus dasar**: setiap pemanggilan rekursif harus membuat masalah benar-benar mengecil, sehingga cepat atau lambat kasus dasar pasti tercapai.

Sebagai pemanasan, perhatikan versi rekursif dari penjumlahan array di Modul 01. Gagasannya: jumlah n bilangan pertama sama dengan jumlah $n - 1$ bilangan pertama ditambah bilangan ke-n.

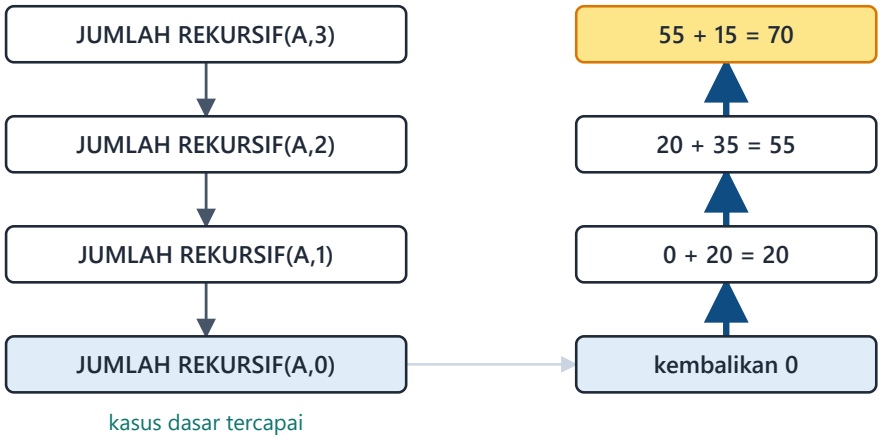
JUMLAH_REKURSIF(A, n) // jumlah A[1..n]

```
1  if n == 0                                  // kasus dasar: array kosong
2      return 0
3  return JUMLAH_REKURSIF(A, n - 1) + A[n]     // kasus rekursif, n mengecil
```

Untuk $A = [20, 35, 15]$, pemanggilan `JUMLAH_REKURSIF(A, 3)` memanggil `JUMLAH_REKURSIF(A, 2)`, lalu `JUMLAH_REKURSIF(A, 1)`, lalu `JUMLAH_REKURSIF(A, 0)` yang langsung mengembalikan 0. Setelah itu hasil "dikembalikan ke atas": $0 + 20 = 20$, $20 + 35 = 55$, $55 + 15 = 70$.

2.3 Tumpukan pemanggilan (call stack)

Ketika sebuah fungsi memanggil fungsi lain (termasuk dirinya sendiri), komputer menyimpan "di mana harus melanjutkan" dan nilai-nilai variabel lokal fungsi pemanggil di sebuah struktur bernama **tumpukan pemanggilan** (*call stack*). Setiap pemanggilan menambah satu lapisan di atas tumpukan. Ketika sebuah pemanggilan selesai (`return`), lapisannya dibuang dan eksekusi kembali ke fungsi yang memanggilnya. Itulah mengapa rekursi membutuhkan memori tambahan sebanding dengan **kedalaman rekursi**.



Gambar 1. Tumpukan pemanggilan JUMLAH_REKURSIF(A, 3). Panah ke bawah: pemanggilan; panah ke atas: nilai yang dikembalikan.

2.4 Membandingkan kedua pendekatan

Aspek	Iteratif	Rekursif
Mekanisme pengulangan	`for`, `while`	Fungsi memanggil dirinya sendiri
Syarat berhenti	Kondisi perulangan menjadi salah	Kasus dasar tercapai
Memori tambahan	Umumnya konstan	Sebanding dengan kedalaman rekursi (call stack)
Kejelasan	Lebih alami untuk proses berurutan	Lebih alami untuk masalah yang strukturnya bersarang (pohon, pembagian masalah)
Risiko umum	Perulangan tak berhenti	Kasus dasar terlupa sehingga terjadi *stack overflow*

Tabel 1. Perbandingan pendekatan iteratif dan rekursif.

PEMBAHASAN INTI

3. Binary Search

3.1 Masalah pencarian

Masalah yang diselesaikan: diberikan array A berisi bilangan yang **sudah terurut dari kecil ke besar**, dengan p sebagai indeks pertama dan n sebagai indeks terakhir. Selidiki apakah suatu bilangan x terdapat di dalam A. Jika ada, kembalikan indeksnya; jika tidak ada, kembalikan `NIL`.

Cara paling sederhana adalah memeriksa elemen satu per satu dari kiri ke kanan (*linear search*). Cara ini bisa membutuhkan n perbandingan. Binary Search memanfaatkan fakta bahwa array sudah terurut sehingga jauh lebih cepat.

3.2 Gagasan utama

Bandingkan x dengan elemen di **tengah** rentang pencarian, yaitu $A[mid]$:

- Jika $x = A[mid]$, pencarian selesai.
- Jika $x > A[mid]$, karena array terurut, x tidak mungkin berada di separuh kiri. Lanjutkan pencarian di separuh kanan ($p = mid + 1$).
- Jika $x < A[mid]$, lanjutkan pencarian di separuh kiri ($n = mid - 1$).

Setiap perbandingan membuang kira-kira separuh elemen yang tersisa. Inilah sumber kecepatannya. Indeks tengah dihitung dengan pembulatan ke bawah:

$$mid = \left\lfloor \frac{p + n}{2} \right\rfloor$$

Notasi $\lfloor y \rfloor$ (*floor*) berarti bilangan bulat terbesar yang tidak melebihi y. Contoh: $\lfloor 3{,}5 \rfloor = 3$, $\lfloor 7/2 \rfloor = 3$, dan $\lfloor 4 \rfloor = 4$.

> **SYARAT MUTLAK**
>
> Binary Search **hanya benar untuk array yang sudah terurut**. Pada array yang tidak terurut, membuang separuh array berdasarkan satu perbandingan tidak lagi sah, sehingga algoritma bisa menyimpulkan bahwa x tidak ada padahal sebenarnya ada.

3.3 Versi iteratif

BINARY-SEARCH(A, x, p, n) // versi iteratif

```
1  while p ≤ n
2      mid = ⌊(p + n) / 2⌋
3      if (x == A[mid])
4          return mid
5      else if x > A[mid]
6          p = mid + 1
7      else n = mid - 1
8  return NIL          // bilangan x tidak ditemukan di dalam array A
```

Variabel p dan n menandai batas kiri dan kanan rentang yang masih mungkin berisi x. Perulangan berhenti dalam dua keadaan: x ditemukan (baris 4), atau rentang menjadi kosong ($p > n$) sehingga algoritma mengembalikan `NIL` (baris 8).

3.4 Versi rekursif

BINARY-SEARCH(A, x, p, n) // versi rekursif

```
1  if p > n
2      return NIL      // kasus dasar 1: rentang kosong, x tidak ditemukan
3  mid = ⌊(p + n) / 2⌋
4  if (x == A[mid])
5      return mid      // kasus dasar 2: x ditemukan
6  else if x > A[mid]
7      return BINARY-SEARCH(A, x, mid + 1, n)
8  else return BINARY-SEARCH(A, x, p, mid - 1)
```

Bandingkan kedua versi. Kondisi perulangan `while p ≤ n` pada versi iteratif berubah menjadi kasus dasar `if p > n` pada versi rekursif. Perintah "geser batas" (`p = mid + 1`) berubah menjadi "panggil diri sendiri dengan batas baru". Kemajuan menuju kasus dasar terjamin karena rentang $[p, n]$ selalu mengecil minimal satu elemen pada setiap pemanggilan.

> **CATATAN PENAMAAN VARIABEL**
>
> Bahan kuliah memakai nama n untuk **indeks terakhir** rentang pencarian, bukan untuk ukuran array, dan nilainya berubah selama pencarian. Agar tidak tertukar, di bagian analisis modul ini ukuran array ditulis sebagai N. Banyak buku menamai batas kanan dengan r atau *high*.

CONTOH DAN PENERAPAN

4. Penelusuran Binary Search

4.1 Pencarian yang gagal: x = 12

Ini adalah latihan dari bahan kuliah: $x = 12$ dan $A = (5, 8, 9, 10, 14, 20)$. Langkah algoritma benar jika keluarannya `NIL`.

indeks	1	2	3	4	5	6
A	5	8	9	10	14	20

Tabel 2. Array A untuk penelusuran.

Putaran	p	n	mid	A[mid]	Keputusan
1	1	6	⌊7/2⌋ = 3	9	12 > 9, maka p = 4
2	4	6	⌊10/2⌋ = 5	14	12 < 14, maka n = 4
3	4	4	⌊8/2⌋ = 4	10	12 > 10, maka p = 5
-	5	4	-	-	p > n, perulangan berhenti, **return NIL**

Tabel 3. Penelusuran versi iteratif untuk x = 12.

Hanya **3 perbandingan** dengan elemen array yang dibutuhkan untuk memastikan 12 tidak ada, padahal array berisi 6 elemen. Pada versi rekursif, nilai-nilai p, n, dan mid persis sama. Bedanya, setiap putaran menjadi satu pemanggilan fungsi baru:

Rantai pemanggilan versi rekursif untuk x = 12

```
BINARY-SEARCH(A, 12, 1, 6)   mid = 3, A[3] = 9  < 12  → panggil (A, 12, 4, 6)
  BINARY-SEARCH(A, 12, 4, 6)   mid = 5, A[5] = 14 > 12  → panggil (A, 12, 4, 4)
    BINARY-SEARCH(A, 12, 4, 4)   mid = 4, A[4] = 10 < 12  → panggil (A, 12, 5, 4)
      BINARY-SEARCH(A, 12, 5, 4)   p > n → return NIL
    ← NIL   ← NIL   ← NIL     (NIL diteruskan ke atas sampai pemanggil pertama)
```

4.2 Pencarian yang berhasil: x = 14

Putaran	p	n	mid	A[mid]	Keputusan
1	1	6	3	9	14 > 9, maka p = 4
2	4	6	5	14	14 == 14, **return 5**

Tabel 4. Penelusuran untuk x = 14 pada array yang sama.



Gambar 2. Rentang pencarian menyusut setiap putaran (x = 12). Kotak gelap: elemen tengah yang dibandingkan.

4.3 Implementasi Python

Kedua versi diterjemahkan ke Python di bawah ini. Array Python dimulai dari indeks 0, sehingga kita menambahkan elemen kosong di posisi 0 agar indeks sama persis dengan pseudocode.

Python: Binary Search iteratif dan rekursif

```python
def binary_search_iter(A, x, p, n):
    while p <= n:
        mid = (p + n) // 2          # // adalah pembagian dengan pembulatan ke bawah
        if x == A[mid]:
            return mid
        elif x > A[mid]:
            p = mid + 1
        else:
            n = mid - 1
    return None                     # None berperan sebagai NIL

def binary_search_rek(A, x, p, n):
    if p > n:
        return None
    mid = (p + n) // 2
    if x == A[mid]:
        return mid
    elif x > A[mid]:
        return binary_search_rek(A, x, mid + 1, n)
    else:
        return binary_search_rek(A, x, p, mid - 1)

A = [None, 5, 8, 9, 10, 14, 20]     # A[1..6]
for x in (12, 14, 5, 20, 1):
    print(x, binary_search_iter(A, x, 1, 6), binary_search_rek(A, x, 1, 6))
```

Keluaran program

```
12 None None
14 5 5
5 1 1
20 6 6
1 None None
```

ANALISIS DAN HUBUNGAN KONSEP

5. Seberapa Cepat Binary Search?

5.1 Banyaknya putaran

Misalkan array berukuran N. Setelah satu perbandingan, rentang yang tersisa paling banyak $\lfloor N/2 \rfloor$ elemen. Setelah dua perbandingan paling banyak sekitar $N/4$, setelah tiga sekitar $N/8$, dan seterusnya. Pencarian pasti berakhir ketika rentangnya kosong. Banyaknya kali kita dapat membagi N dengan 2 sebelum tersisa 1 elemen adalah $\lfloor \lg N \rfloor$, dengan $\lg$ menyatakan logaritma basis 2. Ditambah satu perbandingan terakhir terhadap elemen yang tersisa itu, banyaknya perbandingan pada kasus terburuk adalah

$$\lfloor \lg N \rfloor + 1.$$

Untuk $N = 6$: $\lfloor \lg 6 \rfloor + 1 = \lfloor 2{,}58 \rfloor + 1 = 3$, tepat seperti penelusuran pencarian $x = 12$ di atas.

Ukuran array N	Linear search (N)	Binary search (⌊lg N⌋ + 1)
6	6	3
1.000	1.000	10
1.000.000	1.000.000	20
1.000.000.000	1.000.000.000	30

Tabel 5. Banyaknya perbandingan kasus terburuk: linear search vs binary search.

Menggandakan ukuran array hanya menambah **satu** perbandingan. Dalam notasi yang akan dibahas di Modul 06, waktu kasus terburuk Binary Search adalah $\Theta(\lg N)$, sedangkan linear search $\Theta(N)$. Kasus terbaiknya adalah x tepat berada di tengah pada perbandingan pertama, yaitu 1 perbandingan.

5.2 Memori

Versi iteratif hanya membutuhkan beberapa variabel (p, n, mid), sehingga memorinya konstan. Versi rekursif menumpuk satu lapisan call stack untuk setiap pemanggilan, dan kedalaman rekursinya paling banyak $\lfloor \lg N \rfloor + 2$ (termasuk pemanggilan terakhir yang menemukan rentang kosong). Untuk Binary Search tambahan ini kecil, tetapi pada algoritma lain kedalaman rekursi bisa sebesar N, misalnya `JUMLAH_REKURSIF`. Pada kasus seperti itu versi iteratif lebih aman.

5.3 Hubungan dengan modul lain

- **Running time (Modul 03).** Menghitung berapa kali isi perulangan dijalankan adalah inti analisis running time versi iteratif.
- **Laju pertumbuhan (Modul 05).** Fungsi $\lg n$ tumbuh sangat lambat. Ketika n dinaikkan dari 5 menjadi 25, $\lg n$ hanya naik dua kali lipat.
- **Divide and Conquer dan Recurrence (Modul 08–09).** Binary Search membagi masalah menjadi satu submasalah berukuran separuh. Waktunya memenuhi bentuk recurrence $T(n) = T(n/2) + \Theta(1)$, yang dengan Metode Master menghasilkan $T(n) = \Theta(\lg n)$, sama dengan hasil penalaran di atas.
- **Prasyarat terurut.** Binary Search mengandaikan data sudah terurut. Biaya mengurutkan data (Modul 04 dan 08) sepadan bila pencarian dilakukan berulang kali pada data yang sama.

KESIMPULAN

6. Kesimpulan

- Algoritma iteratif mengulang langkah dengan perulangan; algoritma rekursif mengulang dengan memanggil dirinya sendiri untuk masalah yang lebih kecil.
- Fungsi rekursif yang benar wajib memiliki kasus dasar, kasus rekursif, dan kemajuan yang pasti menuju kasus dasar.
- Rekursi memakai call stack, sehingga butuh memori tambahan sebanding dengan kedalaman rekursi.
- Binary Search mencari x pada array **terurut** dengan membandingkan elemen tengah lalu membuang separuh rentang pada setiap langkah.
- Kasus terburuk Binary Search membutuhkan $\lfloor \lg N \rfloor + 1$ perbandingan, jauh lebih sedikit dibanding N perbandingan pada linear search.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PILIHAN GANDA**

Unsur apa yang **wajib** ada agar sebuah fungsi rekursif pasti berhenti?

A. Perulangan `for` di dalam fungsi.
B. Kasus dasar dan pemanggilan rekursif yang membuat masalah mengecil menuju kasus dasar.
C. Paling sedikit dua pemanggilan rekursif.
D. Parameter berupa array.

2 **PENELUSURAN**

Telusuri Binary Search iteratif untuk $x = 9$ pada $A = (5, 8, 9, 10, 14, 20)$, $p = 1$, $n = 6$. Tuliskan nilai p, n, mid pada setiap putaran dan keluarannya.

3 **PENELUSURAN**

Telusuri Binary Search rekursif untuk $x = 21$ pada array yang sama. Tuliskan rantai pemanggilan fungsinya. Berapa kedalaman rekursi (banyaknya pemanggilan) yang terjadi?

4 **BENAR/SALAH DENGAN ALASAN**

"Binary Search tetap memberikan hasil yang benar bila dijalankan pada array $(10, 3, 8, 1, 6)$ untuk mencari $x = 3$." Benar atau salah? Tunjukkan dengan penelusuran.

5 **PERHITUNGAN**

Sebuah array terurut berisi 5.000 elemen. Berapa banyak perbandingan dengan elemen array yang dibutuhkan Binary Search pada kasus terburuk? Bandingkan dengan linear search.

6 **ANALISIS**

Fungsi berikut dimaksudkan menghitung $n!$ (faktorial). Temukan kesalahannya dan jelaskan akibatnya ketika dijalankan.

FAKTORIAL(n)

```
1  return n * FAKTORIAL(n - 1)
```

7 **PILIHAN GANDA**

Mengapa versi iteratif Binary Search lebih hemat memori daripada versi rekursif?

A. Versi iteratif melakukan lebih sedikit perbandingan.
B. Versi rekursif menyalin seluruh array pada setiap pemanggilan.
C. Versi rekursif menambah satu lapisan call stack untuk setiap pemanggilan, sedangkan versi iteratif hanya memakai beberapa variabel.
D. Versi iteratif tidak memerlukan array terurut.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Tanpa kasus dasar, rekursi tidak memiliki titik berhenti. Tanpa kemajuan menuju kasus dasar, kasus dasar mungkin tidak pernah tercapai. Banyaknya pemanggilan rekursif (C) dan jenis parameter (D) tidak menentukan apakah fungsi berhenti.

Soal 2

Putaran 1: $p = 1$, $n = 6$, $mid = 3$, $A[3] = 9$. Karena $x = 9 = A[3]$, algoritma langsung **mengembalikan 3**. Hanya satu perbandingan yang dibutuhkan; ini adalah kasus terbaik Binary Search.

Soal 3

Rantai pemanggilan: `(1,6)`: mid = 3, $21 > 9$ → `(4,6)`: mid = 5, $21 > 14$ → `(6,6)`: mid = 6, $21 > 20$ → `(7,6)`: $p > n$, kembalikan `NIL`. Terjadi **4 pemanggilan** (3 pemanggilan yang membandingkan elemen ditambah 1 pemanggilan yang mendapati rentang kosong). Hasil akhirnya `NIL`.

Soal 4

Salah. Putaran 1: $p = 1$, $n = 5$, $mid = 3$, $A[3] = 8$. Karena $3 < 8$, maka $n = 2$. Putaran 2: $mid = 1$, $A[1] = 10$. Karena $3 < 10$, maka $n = 0$. Rentang kosong, algoritma mengembalikan `NIL`, padahal 3 ada di indeks 2. Array tidak terurut sehingga keputusan membuang separuh array tidak sah.

Soal 5

$\lfloor \lg 5000 \rfloor + 1$. Karena $2^{12} = 4096 \leq 5000 < 8192 = 2^{13}$, maka $\lfloor \lg 5000 \rfloor = 12$, sehingga kasus terburuknya **13 perbandingan**. Linear search pada kasus terburuk membutuhkan 5.000 perbandingan.

Soal 6

Fungsi tidak memiliki **kasus dasar**. `FAKTORIAL(3)` memanggil `FAKTORIAL(2)`, lalu `FAKTORIAL(1)`, `FAKTORIAL(0)`, `FAKTORIAL(-1)`, dan seterusnya tanpa henti. Dalam praktik program akan gagal karena call stack penuh (*stack overflow*). Perbaikan: tambahkan `if n == 0 return 1` sebelum baris rekursif.

Soal 7

Jawaban: C. Kedua versi melakukan perbandingan yang sama banyak (A salah). Pseudocode rekursif hanya meneruskan batas p dan n, bukan menyalin array (B salah). Keduanya sama-sama mensyaratkan array terurut (D salah).

Referensi

Sumber utama: PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Algoritma Iteratif dan Rekursif) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)

3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.